Name : Muhammad Farhan
Roll no: 25K-0538
Section: BCS-3K
Course: Data Structures – Lab
Lab no: 07
Course Instructor: Miss Fatima Gado
Date of Submission: 08/11/2026

Task 1:

```
#include <iostream>
#include <string>

using namespace std;

class Transaction
{
public:
    string id;
    int time;

    Transaction(string transac = "", int timer = 0) : id(transac), time(timer)
    {}
};

void mergeLogs(Transaction array[], Transaction left_array[], Transaction right_array[], int
left_size, int right_size)
{
    int i = 0, l = 0, r = 0;

    while (l < left_size && r < right_size)
    {
        // <= ensures Counter A gets priority if times are equal
        if (left_array[l].time <= right_array[r].time)
        {
            array[i] = left_array[l];
            i++;
            l++;
        }
        else
        {
            array[i] = right_array[r];
            i++;
            r++;
```

```
        }
    }

    while (l < left_size)
    {
        array[i] = left_array[l];
        i++;
        l++;
    }

    while (r < right_size)
    {
        array[i] = right_array[r];
        i++;
        r++;
    }
}

void findLongestGap(Transaction array[], int size)
{
    int max_gap = -1;
    string id_1 = "";
    string id_2 = "";

    for (int i = 0; i < size - 1; i++)
    {
        int gap = array[i + 1].time - array[i].time;
        if (gap > max_gap)
        {
            max_gap = gap;
            id_1 = array[i].id;
            id_2 = array[i + 1].id;
        }
    }

    cout << "The longest gap is " << max_gap << " minutes, between " << id_1 << " and " << id_2
<< endl;
}

string searchByTime(Transaction array[], int size, int target_time)
{
    for (int i = 0; i < size; i++)
    {
        // Returns the first match found since array is already sorted
        if (array[i].time == target_time)
        {
            return array[i].id;
        }
    }
   return "-1";
}
```

```
int main()
{
    Transaction counter_a[4] =
    {
        Transaction("A1", 540),
        Transaction("A2", 555),
        Transaction("A3", 600),
        Transaction("A4", 660)
    };
    int size_a = 4;

    Transaction counter_b[4] =
    {
        Transaction("B1", 545),
        Transaction("B2", 600),
        Transaction("B3", 610),
        Transaction("B4", 700)
    };
    int size_b = 4;

    int total_size = size_a + size_b;
    Transaction merged_log[8];

    mergeLogs(merged_log, counter_a, counter_b, size_a, size_b);

    cout << "Merged log: ";
    for (int i = 0; i < total_size; i++)
    {
        cout << merged_log[i].id << " " << merged_log[i].time;
        if (i < total_size - 1)
        {
            cout << ", ";
        }
    }
    cout << endl;

    findLongestGap(merged_log, total_size);

    cout << "Searching 600 returns: " << searchByTime(merged_log, total_size, 600) << endl;
    cout << "Searching 650 returns: " << searchByTime(merged_log, total_size, 650) << endl;

    return 0;
}
```

```
❯ g++ task1.cpp -o task1.out && ./task1.out
Merged log: A1 540, B1 545, A2 555, A3 600, B2 600, B3 610, A4 660, B4 700
The longest gap is 50 minutes, between B3 and A4
Searching 600 returns: A3
Searching 650 returns: -1
```

Task 2:

```
#include <iostream>
#include <string>

using namespace std;

class Guest
{
public:
    string name;
    int reg_id;
    int priority_score;
    int seat_number;

    Guest(string guest_name = "", int id = 0, int score = 0)
        : name(guest_name), reg_id(id), priority_score(score), seat_number(0)
    {
    }
};

// merges two halves in descending order
void merge(Guest array[], Guest left_array[], Guest right_array[], int left_size, int right_size)
{
    int i = 0, l = 0, r = 0;

    while (l < left_size && r < right_size)
    {
        // >= keeps it stable so earlier registered guests come first on a tie
        if (left_array[l].priority_score >= right_array[r].priority_score)
        {
            array[i] = left_array[l];
            i++;
            l++;
        }
        else
        {
            array[i] = right_array[r];
            i++;
            r++;
        }
    }

    while (l < left_size)
    {
        array[i] = left_array[l];
        i++;
        l++;
    }

    while (r < right_size)
    {
```

```
        array[i] = right_array[r];
        i++;
        r++;
    }
}

// standard recursive merge sort
void mergeSort(Guest array[], int size)
{
    if (size <= 1)
    {
        return;
    }

    int middle = size / 2;
    Guest* left_array = new Guest[middle];
    Guest* right_array = new Guest[size - middle];

    int i = 0;
    int j = 0;

    for (; i < size; i++)
    {
        if (i < middle)
        {
            left_array[i] = array[i];
        }
        else
        {
            right_array[j] = array[i];
            j++;
        }
    }

    mergeSort(left_array, middle);
    mergeSort(right_array, size - middle);

    merge(array, left_array, right_array, middle, size - middle);

    delete[] left_array;
    delete[] right_array;
}

// assigns seat numbers starting from 1 after sorting
void assignSeats(Guest array[], int size)
{
    for (int i = 0; i < size; i++)
    {
        array[i].seat_number = i + 1;
    }
}
```

```
// search for a guest registration ID and returns their seat number
int searchSeatByRegId(Guest array[], int size, int target_id)
{
    for (int i = 0; i < size; i++)
    {
        if (array[i].reg_id == target_id)
        {
            return array[i].seat_number;
        }
    }
    return -1;
}

int main()
{
    int size = 5;
    Guest guest_list[5] = {
        Guest("Areeba", 104, 85),
        Guest("Hamza", 101, 92),
        Guest("Zain", 105, 85),
        Guest("Hira", 102, 96),
        Guest("Saad", 103, 92)
    };

    // sort by priority descending
    mergeSort(guest_list, size);

    // assign seats 1 to N
    assignSeats(guest_list, size);

    // print sorted guest order with seat numbers
    cout << "Seating arrangement:" << endl;
    for (int i = 0; i < size; i++)
    {
        cout << guest_list[i].name << " (" << guest_list[i].seat_number << ")";
        if (i < size - 1)
        {
            cout << ", ";
        }
    }
    cout << endl << endl;

    // search tests
    cout << "Seat for ID 102 (Hira): " << searchSeatByRegId(guest_list, size, 102) << endl;
    cout << "Seat for ID 104 (Areeba): " << searchSeatByRegId(guest_list, size, 104) << endl;
    cout << "Seat for ID 999 (Non-existent): " << searchSeatByRegId(guest_list, size, 999) << endl;

    return 0;
}
```

```
❯ g++ task2.cpp -o task2.out && ./task2.out
Seating arrangement:
Hira (1), Hamza (2), Saad (3), Areeba (4), Zain (5)

Seat for ID 102 (Hira): 1
Seat for ID 104 (Areeba): 4
Seat for ID 999 (Non-existent): -1
```

Task 3:

```
#include <iostream>
#include <string>

using namespace std;

class Computer
{
public:
    int id;
    int memory;

    Computer(int comp_id = 0, int comp_memory = 0)
        : id(comp_id), memory(comp_memory)
    {
    }
};

void quickSort(Computer array[], int start, int end)
{
    if (start >= end)
    {
        return;
    }

    // pick middle element as pivot
    int middle = start + (end - start) / 2;
    int pivot = array[middle].memory;

    // two scanning pointers from left and right
    int i = start;
    int j = end;

    while (i <= j)
    {
        // move left pointer until an element >= pivot is found
        while (array[i].memory < pivot)
        {
            i++;
        }

        // move right pointer until an element <= pivot is found
        while (array[j].memory > pivot)
```

```
        {
            j--;
        }

        // swap elements and step both pointers forward to prevent infinite loops on matching values
        if (i <= j)
        {
            Computer temp = array[i];
            array[i] = array[j];
            array[j] = temp;
            i++;
            j--;
        }
    }

    // sort left partition
    if (start < j)
    {
        quickSort(array, start, j);
    }

    // sort right partition
    if (i < end)
    {
        quickSort(array, i, end);
    }
}

void printComputers(Computer array[], int size)
{
    for (int i = 0; i < size; i++)
    {
        cout << "Computer " << array[i].id << " (" << array[i].memory << " MB)";
        if (i < size - 1)
        {
            cout << ", ";
        }
    }
    cout << endl;
}

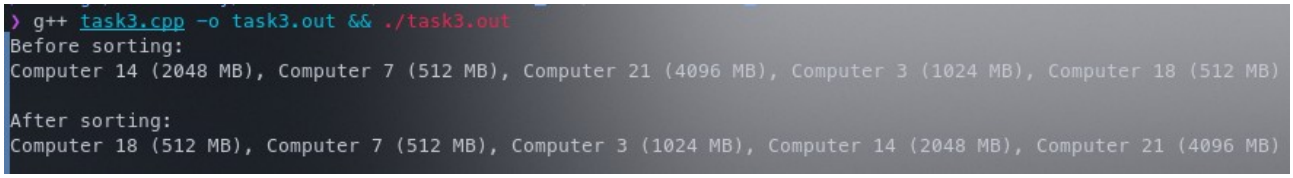
int main()
{
    int size = 5;
    Computer lab_computers[5] = {
        Computer(14, 2048),
        Computer(7, 512),
        Computer(21, 4096),
        Computer(3, 1024),
        Computer(18, 512)
    };
```

```
    cout << "Before sorting:" << endl;
    printComputers(lab_computers, size);

    quickSort(lab_computers, 0, size - 1);

    cout << "\nAfter sorting:" << endl;
    printComputers(lab_computers, size);

    return 0;
}
```



Task 4:

```
#include <iostream>
#include <string>

using namespace std;

int getMax(int arr[], int n)
{
    int max_val = arr[0];
    for (int i = 1; i < n; i++)
    {
        if (arr[i] > max_val)
        {
            max_val = arr[i];
        }
    }
    return max_val;
}

void countingSort(int arr[], int n, int exp)
{
    int* output = new int[n];
    int count[10] = {0};

    // count occurrences of each digit
    for (int i = 0; i < n; i++)
    {
        int digit = (arr[i] / exp) % 10;
        count[digit]++;
    }

    // prefix sum for index positions
    for (int i = 1; i < 10; i++)
    {
```

```
        count[i] += count[i - 1];
    }

    // build output array backwards to keep it stable
    for (int i = n - 1; i >= 0; i--)
    {
        int digit = (arr[i] / exp) % 10;
        output[count[digit] - 1] = arr[i];
        count[digit]--;
    }

    // copy sorted elements back into arr
    for (int i = 0; i < n; i++)
    {
        arr[i] = output[i];
    }

    delete[] output;
}

void radixSort(int arr[], int n)
{
    if (n <= 1)
    {
        return;
    }

    int max_val = getMax(arr, n);

    for (int exp = 1; max_val / exp > 0; exp *= 10)
    {
        countingSort(arr, n, exp);
    }
}

// find all values that tie for the highest frequency
void findMostFrequent(int arr[], int n)
{
    if (n <= 0)
    {
        return;
    }

    int max_freq = 0;
    int current_freq = 1;

    // pass 1, find the highest frequency count
    for (int i = 1; i < n; i++)
    {
        if (arr[i] == arr[i - 1])
        {
            current_freq++;
```

```
        }
        else
        {
            if (current_freq > max_freq)
            {
                max_freq = current_freq;
            }
            current_freq = 1;
        }
    }
    if (current_freq > max_freq)
    {
        max_freq = current_freq;
    }

    // pass 2, print any intensity matching max_freq
    cout << "The most frequently occurring intensity values:" << endl;
    current_freq = 1;

    for (int i = 1; i < n; i++)
    {
        if (arr[i] == arr[i - 1])
        {
            current_freq++;
        }
        else
        {
            if (current_freq == max_freq)
            {
                cout << "Intensity " << arr[i - 1] << "\tFrequency: " << max_freq << endl;
            }
            current_freq = 1;
        }
    }
    if (current_freq == max_freq)
    {
        cout << "Intensity " << arr[n - 1] << "\tFrequency: " << max_freq << endl;
    }
}

void printArray(int arr[], int n)
{
    for (int i = 0; i < n; i++)
    {
        cout << arr[i];
        if (i < n - 1)
        {
            cout << ", ";
        }
    }
    cout << endl;
}
```

```
int main()
{
    // original pixel intensities kept unchanged
    int original_pixels[10] = {120, 45, 200, 45, 120, 255, 45, 200, 10, 120};
    int n = 10;

    // copy so the original array stays untouched
    int sorted_pixels[10];
    for (int i = 0; i < n; i++)
    {
        sorted_pixels[i] = original_pixels[i];
    }

    radixSort(sorted_pixels, n);

    cout << "Sorted pixel intensities:" << endl;
    printArray(sorted_pixels, n);
    cout << endl;

    findMostFrequent(sorted_pixels, n);

    return 0;
}
```

